

License

- **Open Source:** Apache License 2.0

For full details, please see the [LICENSE](#) file in the repository.

Go

```
go get github.com/mikhaildadaev/uuid
```

The latest stable version of `uuid` is **v1.26.11**.

Run Test

```
go test ./...  
go test -bench=. ./...  
go test -cover ./...  
go test -race ./...
```

Key Features

- **UUIDv1..v8 & Null** — Generate all UUID versions and SQL-compatible null UUID;
- **Parse & Validate** — Parse and validate standard and null UUID strings;
- **Metadata Extraction** — Extract timestamp, sequence, node, POSIX, variant, and version;
- **Serialization** — Full Binary, JSON, and Text marshal/unmarshal support;
- **SQL Integration** — Native database/sql driver with Scan/Value interfaces;
- **Zero-Allocation** — Fast, allocator-free core routines on all hot paths.

Limits

- **V2:** `posix` should be `0..255` ;
- **V3/V5:** `name` should be `1..512` symbols *bytlength*;
- **V8:** `node` should be `0..16383` ;

Strings `name` that are 0 bytes or larger than 512 bytes return a null UUID to prevent allocation.

API / Core / Constructors

This page documents how to create every UUID version — from the SQL-compatible `NULL` to the latest timestamp-based V7 and vendor-specific V8. Start with **NewV4** for general use or **NewV7** for sortable identifiers.

NewNull

Creates a UUID that represents a SQL-compatible `NULL` value.

```
un := uuid.NewNull()
fmt.Println(un.IsZero())
fmt.Println(un.String())
```

Output TEXT:

```
true
00000000-0000-0000-0000-000000000000
```

NewV1

Creates a UUID version 1 based on the current timestamp and the local machine's MAC address *or a random node if unavailable*.

```
u1 := uuid.NewV1()
fmt.Println(u1.Version())
```

Output TEXT:

```
1
```

NewV2

Creates a UUID version 2 *DCESecurity* using the current timestamp, the local machine's MAC address, and a POSIX UID/GID.

```
u2 := uuid.NewV2(posix)
fmt.Println(u2.Version())
```

Output TEXT:

```
2
```

NewV3

Creates a UUID version 3 by hashing a namespace identifier and a name with MD5.

```
u3 := uuid.NewV3(uuid.NameSpaceDNS, name)
fmt.Println(u3.Version())
```

Output TEXT:

```
3
```

NewV4

Creates a UUID version 4 using cryptographically secure random numbers.

```
u4 := uuid.NewV4()
fmt.Println(u4.Version())
```

Output TEXT:

4

NewV5

Creates a UUID version 5 by hashing a namespace identifier and a name with SHA-1.

```
u5 := uuid.NewV5(uuid.NameSpaceDNS, name)
fmt.Println(u5.Version())
```

Output TEXT:

5

NewV6

Creates a UUID version 6 *field – compatible with UUIDv1* based on the current timestamp and the local machine's MAC address *or a random node if unavailable*.

```
u6 := uuid.NewV6()
fmt.Println(u6.Version())
```

Output TEXT:

6

NewV7

Creates a UUID version 7 *timestamp – based, lexicographically sortable* using the current Unix timestamp in milliseconds and random bits.

```
u7 := uuid.NewV7()  
fmt.Println(u7.Version())
```

Output TEXT:

```
7
```

NewV8

Creates a UUID version 8 *vendor — specific, custom* using a node identifier and the current timestamp.

```
u8 := uuid.NewV8(node)  
fmt.Println(u8.Version())
```

Output TEXT:

```
8
```

API / Core / Methods

This page documents all instance methods available on UUID and NullUUID — from basic operations like **String** and **Version** to advanced metadata extraction like **Timestamp**, **Node**, and **Sequence**. Every method works with all UUID versions, V1 through V8.

NULLUUID

IsZero

Returns `true` if the NullUUID is invalid *SQLNULL* or the underlying UUID is the zero value.

```
var nu uuid.NullUUID
fmt.Println(nu.IsZero())
nu.Scan("01968727-8c7e-8000-87cb-bdba4f634d9f")
fmt.Println(nu.IsZero())
```

Output TEXT:

```
true
false
```

String

Returns the canonical text form 8 – 4 – 4 – 4 – 12 for a valid UUID, or 00000000-0000-0000-0000-000000000000 for a null value.

```
var nu uuid.NullUUID
fmt.Println(nu.String())
nu.Scan("01968727-8c7e-8000-87cb-bdba4f634d9f")
fmt.Println(nu.String())
```

Output TEXT:

```
00000000-0000-0000-0000-000000000000
01968727-8c7e-8000-87cb-bdba4f634d9f
```

Validate

Validates the NullUUID. Returns `nil` for both null and valid UUIDs *null is considered valid in SQL context*. Returns an error only if the UUID is present but malformed.

```
var nu uuid.NullUUID
fmt.Println(nu.Validate())
nu.Scan("01968727-8c7e-8000-87cb-bdba4f634d9f")
fmt.Println(nu.Validate())
```

Output TEXT:

```
<nil>
<nil>
```

UUID

Bytes

Returns the UUID as a 16-byte slice.

```
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
uu, err := uuid.Parse(uuidV8String)
if err != nil {
    fmt.Println(err)
}
fmt.Printf("%x\n", uu.Bytes())
```

Output TEXT:

```
019687278c7e800087cbbdba4f634d9f
```

Equal

Compares two UUIDs for equality. Returns true if both UUIDs represent the same value.

```
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
uu, err := uuid.Parse(uuidV8String)
if err != nil {
    fmt.Println(err)
}
other := uu
fmt.Println(uu.Equal(other))
```

Output TEXT:

```
true
```

Info

Returns a human-readable summary of the UUID: version, variant, timestamp, sequence, node, and POSIX.

```
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
uu, err := uuid.Parse(uuidV8String)
if err != nil {
    fmt.Println(err)
}
fmt.Println(uu.Info())
```

Output TEXT:

```
UUID: 01968727-8c7e-8000-87cb-bdba4f634d9f
VAR.: RFC4122
VER.: 8
FORM: TTTTTTTT-TTTT-8SSS-VNNN-RRRRRRRRRRRR
INFO: TIME (1-milliseconds interval since 1970-01-01) + SEQUENCE (0-4095) + NOI
TIME: 1746024238206 [2025-04-30 14:43:58.206 +0000 UTC]
SEQ.: 0
NODE: 1995
RAND: bdba4f634d9f
```

IsZero

Returns true if the UUID is the zero value *all 16 bytes are zero*.

```
uu := uuid.NewNull()
fmt.Println(uu.IsZero())
```

Output TEXT:

```
true
```

Node

Returns the node identifier for UUID versions V1, V2, V6, and V8. Returns 0 for other versions.

```
uuidNode := 1995
u8 := uuid.NewV8(uuidNode)
fmt.Println(u8.Node())
```

Output TEXT:

```
1995
```

Posix

Returns the POSIX UID/GID for UUID version V2. Returns 0 for other versions.

```
uuidPosType := 0
uuidPosValue := 501
u2 := uuid.NewV2(uuidPosType, uuidPosValue)
fmt.Println(u2.Posix())
```

Output TEXT:

```
UID 501
```

Sequence

Returns the clock sequence field for UUID versions V1, V2, V6, and V7. Returns 0 for other versions.

```
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
uu, err := uuid.Parse(uuidV8String)
if err != nil {
    fmt.Println(err)
}
fmt.Println(uu.Sequence())
```

Output TEXT:

0

String

Returns the UUID in its canonical text form: 8-4-4-4-12 hex digits with hyphens.

```
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
uu, err := uuid.Parse(uuidV8String)
if err != nil {
    fmt.Println(err)
}
fmt.Println(uu.String())
```

Output TEXT:

01968727-8c7e-8000-87cb-bdba4f634d9f

Timestamp

Returns the timestamp embedded in UUID versions V1, V2, V6, and V7. Returns a zero time for other versions.

```
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
uu, err := uuid.Parse(uuidV8String)
if err != nil {
    fmt.Println(err)
}
fmt.Println(uu.Timestamp())
```

Output TEXT:

1746024238206

Validate

Validates the UUID and returns the UUID version on success, or an error on failure.

```
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
uu, err := uuid.Parse(uuidV8String)
if err != nil {
    fmt.Println(err)
}
fmt.Println(uu.Validate())
```

Output TEXT:

```
<nil>
```

Variant

Returns the UUID variant number. Valid UUIDs return 1 *RFC4122standard*.

```
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
uu, err := uuid.Parse(uuidV8String)
if err != nil {
    fmt.Println(err)
}
fmt.Println(uu.Variant())
```

Output TEXT:

```
1
```

Version

Returns the UUID version number *1through8*. Returns 0 for a null UUID.

```
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
uu, err := uuid.Parse(uuidV8String)
if err != nil {
    fmt.Println(err)
}
fmt.Println(uu.Version())
```

Output TEXT:

```
8
```

API / Marshal / Methods

This page documents serialization and deserialization of UUIDs to and from **Binary**, **Json**, and **Text** formats. Every marshal/unmarshal method works with all UUID versions — V1 through V8 and Null.

NULLUUID

MarshalBinary

Encodes the NullUUID into its 16-byte binary representation. Returns all zeros for a null value.

```
var nu uuid.NullUUID
data, err := nu.MarshalBinary()
if err != nil {
    fmt.Println(err)
}
fmt.Printf("%x\n", data)
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
nu.Scan(uuidV8String)
data, err = nu.MarshalBinary()
if err != nil {
    fmt.Println(err)
}
fmt.Printf("%x\n", data)
```

Output TEXT:

```
019687278c7e800087cbbdba4f634d9f
```

MarshalJson

Encodes the NullUUID into a Json string. Returns null for a null value.

```
var nu uuid.NullUUID
data, err := nu.MarshalJson()
if err != nil {
    fmt.Println(err)
}
fmt.Println(string(data))
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
nu.Scan(uuidV8String)
data, err := nu.MarshalJson()
if err != nil {
    fmt.Println(err)
}
fmt.Println(string(data))
```

Output TEXT:

```
null
"01968727-8c7e-8000-87cb-bdba4f634d9f"
```

MarshalText

Encodes the NullUUID into its canonical text form. Returns an empty string for a null value.

```

var nu uuid.NullUUID
data, err := nu.MarshalText()
if err != nil {
    fmt.Println(err)
}
fmt.Println(string(data))
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
nu.Scan(uuidV8String)
data, err := nu.MarshalText()
if err != nil {
    fmt.Println(err)
}
fmt.Println(string(data))

```

Output TEXT:

```

01968727-8c7e-8000-87cb-bdba4f634d9f

```

UnmarshalBinary

Decodes a NullUUID from its 16-byte binary representation. Sets Valid based on the input.

```

var nu uuid.NullUUID
data := []byte{}
if err := nu.UnmarshalBinary(data); err != nil {
    fmt.Println(err)
}
fmt.Println("Valid:", nu.Valid)
fmt.Println("UUID:", nu.UUID)
uuidV8Binary := []byte{0x01, 0x96, 0x87, 0x27, 0x8c, 0x7e, 0x80, 0x00, 0x87, 0x00, 0xb4, 0xdb, 0x4f, 0x63, 0x4d, 0x9f}
err := nu.UnmarshalBinary(uuidV8Binary)
if err != nil {
    fmt.Println(err)
}
fmt.Println("Valid:", nu.Valid)
fmt.Println("UUID:", nu.UUID)

```

Output TEXT:

```
Valid: false
UUID: 00000000-0000-0000-0000-000000000000
Valid: true
UUID: 01968727-8c7e-8000-87cb-bdba4f634d9f
```

UnmarshalJson

Decodes a NullUUID from a Json-encoded string. Accepts null for SQL NULL or a valid UUID string.

```
var nu uuid.NullUUID
data := []byte(`null`)
if err := nu.UnmarshalJson(data); err != nil {
    fmt.Println(err)
}
fmt.Println("Valid:", nu.Valid)
fmt.Println("UUID:", nu.UUID)
uuidV8Json := []byte(`"01968727-8c7e-8000-87cb-bdba4f634d9f"`)
err := nu.UnmarshalJson(uuidV8Json)
if err != nil {
    fmt.Println(err)
}
fmt.Println("Valid:", nu.Valid)
fmt.Println("UUID:", nu.UUID)
```

Output TEXT:

```
Valid: false
UUID: 00000000-0000-0000-0000-000000000000
Valid: true
UUID: 01968727-8c7e-8000-87cb-bdba4f634d9f
```

UnmarshalText

Decodes a NullUUID from its canonical text form. Accepts an empty string for SQL NULL.

```

var nu uuid.NullUUID
data := []byte("")
if err := nu.UnmarshalText(data); err != nil {
    fmt.Println(err)
}
fmt.Println("Valid:", nu.Valid)
fmt.Println("UUID:", nu.UUID)
uuidV8Text := []byte("01968727-8c7e-8000-87cb-bdba4f634d9f")
err := nu.UnmarshalText(uuidV8Text)
if err != nil {
    fmt.Println(err)
}
fmt.Println("Valid:", nu.Valid)
fmt.Println("UUID:", nu.UUID)

```

Output TEXT:

```

Valid: false
UUID: 00000000-0000-0000-0000-000000000000
Valid: true
UUID: 01968727-8c7e-8000-87cb-bdba4f634d9f

```

UUID

MarshalBinary

Encodes the UUID into its 16-byte binary representation.

```

uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
uu, err := uuid.Parse(uuidV8String)
if err != nil {
    fmt.Println(err)
}
data, err := uu.MarshalBinary()
if err != nil {
    fmt.Println(err)
}
fmt.Printf("%x\n", data)

```

Output TEXT:

```
019687278c7e800087cbbdba4f634d9f
```

MarshalJson

Encodes the UUID into a Json string in the standard 8-4-4-4-12 format.

```
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
uu, err := uuid.Parse(uuidV8String)
if err != nil {
    fmt.Println(err)
}
data, err := uu.MarshalJson()
if err != nil {
    fmt.Println(err)
}
fmt.Println(string(data))
```

Output TEXT:

```
"01968727-8c7e-8000-87cb-bdba4f634d9f"
```

MarshalText

Encodes the UUID into its canonical text form: 32 hex digits with hyphens.

```
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
uu, err := uuid.Parse(uuidV8String)
if err != nil {
    fmt.Println(err)
}
data, err := uu.MarshalText()
if err != nil {
    fmt.Println(err)
}
fmt.Println(string(data))
```

Output TEXT:

```
01968727-8c7e-8000-87cb-bdba4f634d9f
```

UnmarshalBinary

Decodes a UUID from its 16-byte binary representation. An error is returned if the input is not exactly 16 bytes long.

```
var uu uuid.UUID
uuidV8Binary := []byte{0x01, 0x96, 0x87, 0x27, 0x8c, 0x7e, 0x80, 0x00, 0x87, 0x27, 0x8c, 0x7e, 0x80, 0x00, 0x87, 0x27}
err := uu.UnmarshalBinary(uuidV8Binary)
if err != nil {
    fmt.Println(err)
}
fmt.Println(uu.String())
```

Output TEXT:

```
01968727-8c7e-8000-87cb-bdba4f634d9f
```

UnmarshalJson

Decodes a UUID from a Json-encoded string. Accepts both quoted and unquoted forms; returns an error if the format is invalid.

```
var uu uuid.UUID
uuidV8Json := []byte(`"01968727-8c7e-8000-87cb-bdba4f634d9f"`)
err := uu.UnmarshalJson(uuidV8Json)
if err != nil {
    fmt.Println(err)
}
fmt.Println(uu.String())
```

Output TEXT:

```
01968727-8c7e-8000-87cb-bdba4f634d9f
```

UnmarshalText

Decodes a UUID from its canonical text form *32hexdigitswithhyphens*. Case-insensitive; returns an error if the format is invalid.

```
var uu uuid.UUID
uuidV8Text := []byte("01968727-8c7e-8000-87cb-bdba4f634d9f")
err := uu.UnmarshalText(uuidV8Text)
if err != nil {
    fmt.Println(err)
}
fmt.Println(uu.String())
```

Output TEXT:

```
01968727-8c7e-8000-87cb-bdba4f634d9f
```

API / SQL / Methods

This page documents the native `database/sql` integration for UUIDs. Both `Scan` and `Value` methods make UUID work seamlessly with SQL databases, including `NULL` support for nullable columns.

NULLUUID

Scan

Implements the `sql.Scanner` interface for nullable UUID columns. Accepts `nil` for SQL NULL or a valid UUID string/bytes. Sets `Valid` to `true` when a UUID is present, `false` for NULL.

```

var nu uuid.NullUUID
if err := nu.Scan(nil); err != nil {
    fmt.Println(err)
}
fmt.Println("Valid:", nu.Valid)
fmt.Println("UUID:", nu.UUID)
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
if err := nu.Scan(uuidV8String); err != nil {
    fmt.Println(err)
}
fmt.Println("Valid:", nu.Valid)
fmt.Println("UUID:", nu.UUID)

```

Output TEXT:

```

Valid: false
UUID: 00000000-0000-0000-0000-000000000000
Valid: true
UUID: 01968727-8c7e-8000-87cb-bdba4f634d9f

```

Value

Implements the `driver.Valuer` interface for nullable UUID columns. Returns `nil` when `Valid` is `false`, or the UUID string when `Valid` is `true`.

```

var nu uuid.NullUUID
value, _ := nu.Value()
fmt.Println(value)
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
nu.Scan(uuidV8String)
value, _ = nu.Value()
fmt.Println(value)

```

Output TEXT:

```

<nil>
01968727-8c7e-8000-87cb-bdba4f634d9f

```

UUID

Scan

Implements the `sql.Scanner` interface. Decodes a UUID from a database value — accepts string, byte slice, or nil *NULL*. Returns an error if the value cannot be parsed.

```
var uu uuid.UUID
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
err := uu.Scan(uuidV8String)
if err != nil {
    fmt.Println(err)
}
fmt.Println(uu.String())
```

Output TEXT:

```
01968727-8c7e-8000-87cb-bdba4f634d9f
```

Value

Implements the `driver.Valuer` interface. Encodes the UUID into a value suitable for database storage. Returns `nil` for a null UUID.

```
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
uu, err := uuid.Parse(uuidV8String)
if err != nil {
    fmt.Println(err)
}
value, err := uu.Value()
if err != nil {
    fmt.Println(err)
}
fmt.Println(value)
```

Output TEXT:

```
01968727-8c7e-8000-87cb-bdba4f634d9f
```

